Gen AI 시대의 파이썬: AI 하네싱 특강

Antigravity로 배우는 실전 바이브 코딩

이동현

2026년 4월

한국외국어대학교 Social Science & AI융합학부

오늘의 목표

- **전반 복습**: 8주간 배운 전체 과정을 통합 회고합니다.

오늘의 목표

- **전반 복습**: 8주간 배운 전체 과정을 통합 회고합니다.
- **후반 실습**: AI 코딩 어시스턴트를 **효과적으로 활용**하여 웹 서비스를 만듭니다.

오늘의 목표

- **전반 복습**: 8주간 배운 전체 과정을 통합 회고합니다.
- **후반 실습**: AI 코딩 어시스턴트를 **효과적으로 활용**하여 웹 서비스를 만듭니다.
- "**바이브 코딩**(Vibe Coding)" — 방향만 잡으면 AI가 구현하는 시대!

오늘의 목표

- **전반 복습**: 8주간 배운 전체 과정을 통합 회고합니다.
- **후반 실습**: AI 코딩 어시스턴트를 **효과적으로 활용**하여 웹 서비스를 만듭니다.
- "**바이브 코딩**(Vibe Coding)" — 방향만 잡으면 AI가 구현하는 시대!
- Antigravity의 에이전트 루프 시스템으로 **품질을 스스로 개선**하는 AI 활용법을 실습합니다.

세션 시간표

세션	시간	주요 내용
5	14:20 – 15:10	8주 전체 복습 (1)
6	15:20 – 16:10	8주 전체 복습 (2)
7	16:20 – 17:10	AI 하네싱 개념 & Antigravity 실전 바이브 코딩
8	17:20 – 18:10	[실습] Antigravity 에이전트 루프로 미니 웹앱 만들기 + 정리

전반 (세션 5–6)

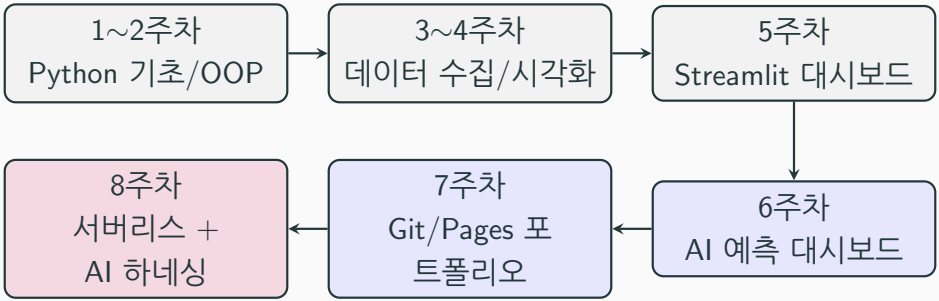
8주 과정 **전체 복습**
Python → 데이터 → 웹 → 배포

후반 (세션 7–8)

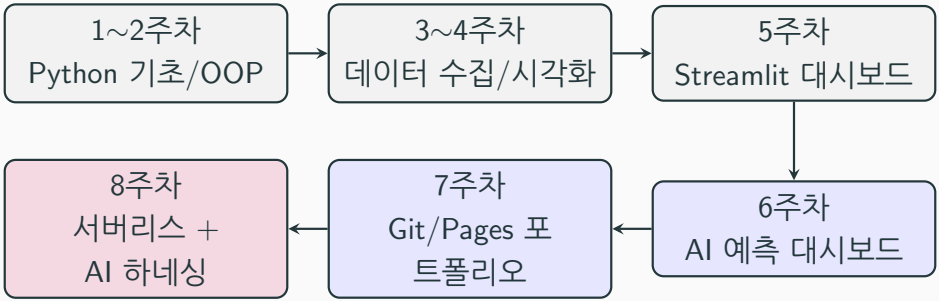
AI 하네싱 **개념 + 실습**
프롬프트 → 에이전트 루프 → 실전

세션 5: 8주 전체 복습 (1)

우리의 8주 여정


1~2주차
Python 기초/OOP
3~4주차
데이터 수집/시각화
5주차
Streamlit 대시보드
6주차
AI 예측 대시보드
7주차
Git/Pages 포
트폴리오
8주차
서버리스 +
AI 하네싱

우리의 8주 여정



오늘의 위치

여러분은 이제 **데이터를 다루고 분석**하는 것부터 **웹서비스 배포와 자동화**까지 혼자서 해낼 수 있습니다!

복습 1 — Python 기초와 데이터 분석 (1–5주차)

주차	핵심 내용	배운 도구/개념
1~2주차	Python 기초, OOP	변수, 함수, 클래스, 상속
3주차	데이터 수집/전처리	Pandas, API 호출, JSON
4주차	데이터 시각화	Matplotlib, Seaborn
5주차	시계열 분석 + 대시보드	Streamlit, 이동 평균, 분해

핵심 파이프라인

데이터 수집(API) → 전처리(Pandas) → 시각화(Matplotlib) → 대시보드(Streamlit)

복습 (1) — AI/ML 예측 (6주차)

학습한 내용

- 머신러닝 3유형 (지도/비지도/강화)
- scikit-learn 선형 회귀
- LSTM 딥러닝 시계열 예측
- 모델 로드 및 추론

핵심 워크플로우

1. 데이터 준비 (슬라이딩 윈도우)
2. Train/Test 분할
3. 모델 학습 (`fit`)
4. 예측 (`predict`)
5. 성능 평가 (RMSE, R^2)

핵심 교훈

AI를 "만드는 사람"이 아닌 "활용하는 사람"으로서, 학습된 모델을 로드하고 예측하고 시각화!

세션 6: 8주 전체 복습 (2)

복습 2 — Git/GitHub과 포트폴리오 (7주차)

Git/GitHub 핵심

- 버전 관리의 개념
- `clone`, `commit`, `push`
- Fork 및 Pull Request
- GitHub Pages 배포

포트폴리오 웹사이트

- AstroWind 테마 활용
- Markdown으로 포스트 작성
- CI/CD 자동 배포
- 사용자`.github.io` 도메인

복습 (2) — 서버 없이 백엔드 만들기 (8주차 오전)

완전 무료 풀스택

호스팅(Pages) + DB(Issues) + 타이머 서버(Actions) = **비용 0원 ($0)**

복습 (2) — 전체 파이프라인 통합

여러분이 8주간 만든 것

1. **Python 기초**: 데이터를 다루는 프로그래밍 능력
2. **데이터 분석**: 수집 → 전처리 → 시각화 파이프라인
3. **대시보드**: Streamlit으로 인터랙티브 앱 제작
4. **AI 예측**: 사전학습 모델 로드 및 추론
5. **포트폴리오**: Git/GitHub Pages로 웹사이트 배포
6. **서버리스**: Issues DB + Actions 자동화

다음 단계: AI 하네싱

이 모든 기초 위에, **AI 코딩 어시스턴트**를 활용하면 생산성이 10배 이상 올라갑니다!

세션 7: AI 하네싱 개념

패러다임의 전환

구분	기존 코딩 방식	AI 하네싱 방식
코드 작성	직접 한 줄씩 타이핑	AI에게 의도를 설명
디버깅	에러 → 구글 검색	AI가 분석 및 수정 제시
디자인	피그마 → CSS 수작업	AI가 즉시 코드로 구현
학습	문법 암기 → 응용	목표 설명 → 결과 확인

핵심 인사이트

8주간 배운 기초는 "코드를 더 잘 읽고, AI 결과물을 더 잘 검증할 수 있는 눈"이 됩니다.

“바이브 코딩(Vibe Coding)”이란?

Andrej Karpathy (OpenAI 공동창업자)

“코드에 몰입하지 않고, **방향(vibe)**만 잡으면 AI가 구현한다.”

- 개발자 = **감독(Director)**
- AI = **실행자(Builder)**
- 핵심 역할: 무엇을 만들지 결정하기

“바이브 코딩(Vibe Coding)”이란?

Andrej Karpathy (OpenAI 공동창업자)

“코드에 몰입하지 않고, **방향(vibe)**만 잡으면 AI가 구현한다.”

- 개발자 = **감독(Director)**
- AI = **실행자(Builder)**
- 핵심 역할: 무엇을 만들지 결정하기

우리의 새로운 포지션

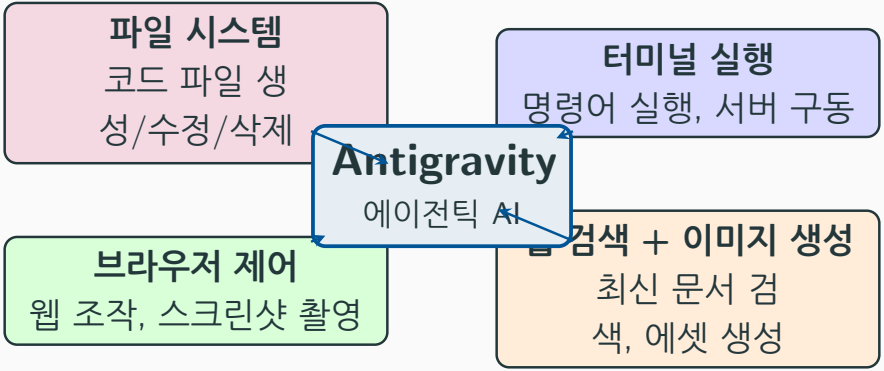
“코딩을 못해도 된다”가 아닙니다.

“코딩을 알면 AI를 **더 잘 쓸 수 있다**”입니다!

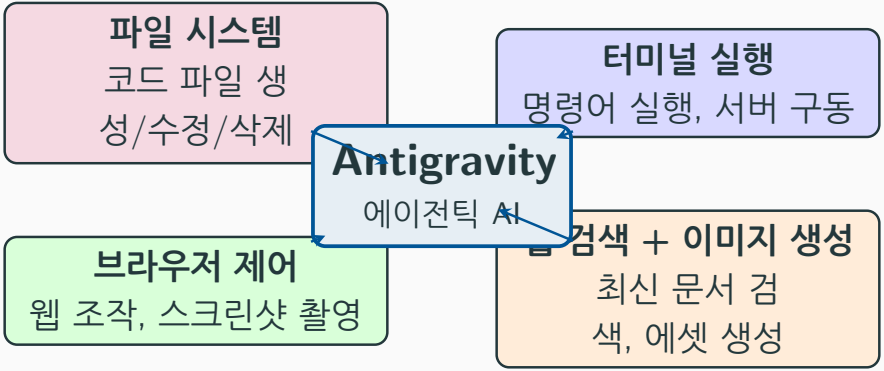
AI 코딩 어시스턴트 생태계

도구	제작사	핵심 특징
GitHub Copilot	GitHub/MS	에디터 내 코드 자동완성, 가장 보편적
Cursor	Cursor Inc.	AI 네이티브 에디터, 코드베이스 이해 탁월
Claude Code	Anthropic	터미널 기반, 대규모 리팩토링에 강점
Antigravity	**Google DeepMind**	**브라우저 + 파일 + 이미지 생성 통합**
Windsurf	Codium	에디터 내장형, Flow 기반 대화

Antigravity가 특별한 이유



Antigravity가 특별한 이유



핵심 차별점

단순 코드 생성을 넘어, **파일 생성** → **서버 실행** → **브라우저 확인** → **수정**까지 전 과정을 AI가 자율적으로 수행합니다.

라이브 데모: 10초 만에 웹페이지 만들기

시연 시나리오

1. Antigravity에게 요청:
   "다크 테마의 자기소개 랜딩 페이지를 만들어줘"
2. AI가 **자동으로**:
   - HTML/CSS/JS 파일 생성
   - 로컬 서버 구동
   - 브라우저에서 결과 확인
3. 수정 요청: "히어로 섹션에 그라디언트 배경 추가해줘"

수강생이 배울 점

AI에게 요청하는 것만으로 **즉시 동작하는 결과물**이 나옵니다.

핵심은 어떻게 요청하느냐!

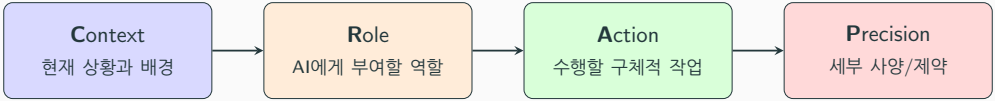
좋은 프롬프트 vs 나쁜 프롬프트

× 나쁜 프롬프트	✓ 좋은 프롬프트
"웹사이트 만들어줘"	"다크 모드 기반 포트폴리오 랜딩 페이지를 만들어줘. Hero 섹션에 이름과 소개, 프로젝트 카드 3개를 포함해줘."
"에러 고쳐줘"	"이 JS 코드의 `TypeError` 에러를 수정해줘. `issues` 변수가 `null`일 때 빈 배열로 대체하도록."
"더 예쁘게"	"카드에 글라스모피즘 효과를 적용하고, hover 시 `scale(1.03)` 변환과 그림자를 추가해줘."

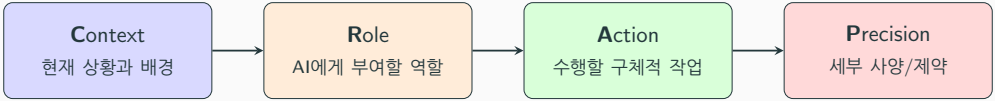
핵심 원칙

구체적일수록 AI의 결과물 품질이 올라갑니다. "예쁘게" 대신 "어떤 스타일로"를 말해주세요.

프롬프트 작성 공식 — CRAP



프롬프트 작성 공식 — CRAP



예시

"**[C]** @food-board 폴더의 기존 프로젝트를 기반으로, **[R]** 시니어 프론트엔드 개발자로서, **[A]** 맛집 카드에 별점 기능을 추가해줘. **[P]** 1~5 별 선택 가능, 노란색 별 아이콘, hover 시 미리보기"

@멘션과 컨텍스트 활용

@멘션 기능

특정 파일/폴더를 AI에게 직접 지정:

- `@index.html` — 이 파일 수정
- `@food-board/` — 이 폴더 전체 참조
- `@style.css` — CSS만 수정

컨텍스트 유지

대화가 이어지면 AI가 이전 맥락을 기억합니다:

1. "카드 UI 만들어줘"
2. "카드에 이미지 추가" ← 이전 맥락 유지
3. "hover 효과도 넣어줘" ← 누적

팁

AI에게 **에러 메시지**를 그대로 복사해서 전달하면 자동으로 원인을 분석하고 수정합니다!

반복 개선(Iteration) 전략

1차 프롬프트 → 결과물 확인

↓

수정 요청: "헤더 색상을 그라디언트로 변경"

↓

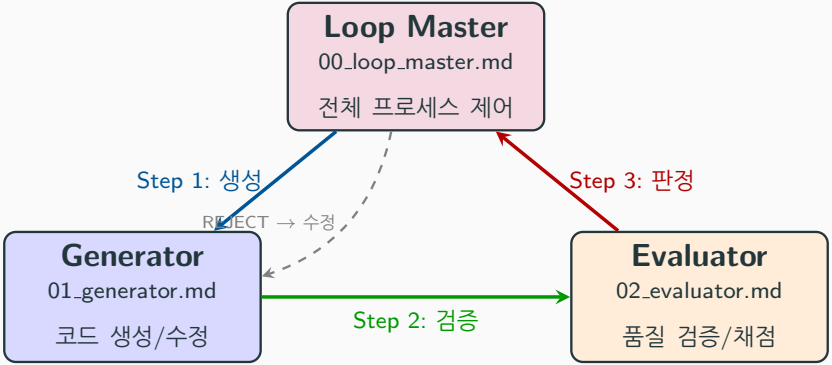
추가 요청: "모바일 반응형 적용"

↓

완성! 점진적으로 목표에 도달

세션 8: 실습 및 정리

에이전트 루프란?



핵심 아이디어

AI가 코드를 생성한 뒤, **스스로 품질을 평가하고 수정**하는 자동화된 피드백 루프입니다.

에이전트 루프 구성 파일

파일	역할	하는 일
`00_loop_master.md`	마스터 프로세스	루프 제어, 종료 조건, 로그 관리
`01_generator.md`	생성자 (Builder)	코드 작성, 디자인 원칙 적용
`02_evaluator.md`	평가자 (Critic)	100점 만점 루브릭으로 채점

핵심 포인트

이 3개 파일을 **프로젝트 폴더**에 넣어두면, Antigravity가 이 지침을 읽고 **하나의 에이전트 안에서** 생성자 역할과 평가자 역할을 번갈아 수행합니다.

하나의 에이전트, 두 개의 역할

실제 동작 원리

Antigravity는 **하나의 에이전트**가 3개 MD 파일을 모두 읽고, 상황에 따라 "모자를 바꿔 쓰듯" 역할을 전환합니다:

1. `01_generator.md`를 따라 코드 생성
2. `02_evaluator.md`를 따라 자기 검증
3. 기준 미달 시 다시 생성자로 복귀

왜 분리하나요?

역할을 **MD 파일로 분리**하면:

- 생성 규칙과 평가 기준이 섞이지 않음
- 각 역할의 지침을 독립적으로 수정 가능
- AI가 "내가 만든 것을 내가 평가"하는 자기 편향을 루브릭으로 억제

비유

영화 감독이 촬영 후 편집실에서 자기 영상을 냉정하게 평가하는 것과 같습니다.

"생성자 모자" → "평가자 모자"로 갈아쓰면 품질을 높입니다.

싱글 에이전트 vs 멀티 에이전트

구분	**싱글 에이전트** (오늘 실습)	**멀티 에이전트**
구조	1개 세션이 3개 MD 파일을 모두 읽고 역할 전환	세션 A(생성자), 세션 B(평가자)를 별도로 운영
자동화	✓ **완전 자동** (프롬프트 1번)	× 사람이 결과를 복사해 전달
편향 차단	△ 루브릭으로 억제	✓ **완전 차단** (독립 세션)
추천	소규모/실습/빠른 프로토타입	대규모/팀/품질 극대화

핵심 인사이트

`00_loop_master.md`가 바로 "**멀티 에이전트의 자동화 버전**"입니다.
하나의 에이전트가 생성자 → 평가자 역할을 **자동으로 번갈아** 수행하므로, 사람 개입 없이 품질 루프가 돌아갑니다.

멀티 에이전트: 수동이지만 더 강력한 방법

Step-by-Step

1. Antigravity에서 **New Chat 2개** 열기
2. **세션 A**: "`@01_generator.md`를 읽고 소감 보드를 만들어줘"
3. **세션 B**: "`@02_evaluator.md`를 읽고 `@project/`를 채점해줘"
4. 세션 B 결과를 **복사** → 세션 A에 **붙여넣기**
5. 90점 이상까지 반복

왜 수동인가?

각 대화 세션은 **독립된 프로세스**이기 때문에 세션 간 자동 통신이 불가합니다.

하지만 그 덕분에:
→ 생성한 AI ≠ 평가하는 AI
→ **자기 편향이 완전히 차단**됩니다.

결론: 오늘 실습은 싱글 에이전트

프롬프트 1번으로 자동 루프가 도는 싱글 에이전트 방식을 실습합니다.
멀티 에이전트는 품질을 더 높이고 싶을 때 **선택적으로** 사용하세요.

Loop Master — 마스터 프로세스 핵심

에이전트 루프 SOP (4단계)

Step 1 – Generate 사용자의 요청을 받으면 `01_generator.md` 규칙을 적용해 웹페이지를 작성

Step 1.5 – Validate 브라우저에서 실제로 열어 렌더링을 확인 (깨진 페이지는 채점 불가)

Step 2 – Evaluate `02_evaluator.md`의 정량적 루브릭(100점 만점)으로 채점

Step 3 – Revise REJECT 피드백을 수용하여 리팩토링 (이전 장점은 유지)

종료 조건

최대 **3회** 반복 또는 총점 **90점 이상** 달성 시 APPROVE

Evaluator — 100점 만점 루브릭

#	평가 축	배점	핵심 평가 기준
1	시각적 완성도	0–25	프리미엄 디자인, 색상 조화, 타이포그래피
2	인터랙션 품질	0–25	부드러운 애니메이션, 60fps, 호버 효과
3	코드 품질	0–25	시맨틱 HTML, CSS 변수, 성능 최적화
4	UX 설계	0–25	반응형, 네비게이션, 가독성
	합계	**/100**	

APPROVE 조건

최소 1회 REJECT 경험 후
총점 **90점 이상**

Anti-Bias Rule

1차 초안에 90점 이상 부여 **금지**
(LLM 자기 편향 방지)

에이전트 루프 실행 흐름

Cycle 1: Generator가 초안 생성 → Evaluator 채점: 72/100 [REJECT]

↓

Cycle 2: 피드백 반영 수정 → Evaluator 재채점: 86/100 [REJECT]

↓

Cycle 3: 세부 개선 완료 → Evaluator 최종: 93/100 [APPROVE]

↓

완료: 최종 스코어카드 + 수정 이력 요약 + 최종 아티팩트 출력

핵심

사람이 "더 예쁘게 해줘"를 반복할 필요 없이, AI가 **스스로 기준에 도달할 때까지**

실습 목표

“한 줄 소감 보드” 만들기

- 8주차에 배운 GitHub Issues API 활용
- 글라스모피즘 다크 모드 UI
- 에이전트 루프로 **자동 품질 개선**
- GitHub Pages로 배포

에이전트 루프 파일 준비

프로젝트 폴더에 3개 MD 파일을 배치한 뒤 Antigravity에게 요청하면 됩니다!

실습 단계별 가이드

단계	수강생의 프롬프트	예상 결과
1	"에이전트 루프 MD 파일을 읽고, GitHub Issues API 기반 소감 보드를 만들어줘"	3개 파일
2	(에이전트가 자동으로 Evaluator 실행)	자체 채점
3	(에이전트가 자동으로 피드백 반영 수정)	재채점: 9
4	"GitHub Pages로 배포해줘"	배포 가이

핵심 체험

수강생은 **1번의 요청**만으로 AI가 **여러 번 자체 개선**하여 완성도 높은 결과물을 받습니다!

실습: 에이전트 루프 시작하기

Step 1 — 프로젝트 폴더에 에이전트 파일 배치

```
antigravity-harnessing/
├── agents/                    ← 에이전트 지침서
│   ├── 00_loop_master.md      ← 마스터 프로세스
│   ├── 01_generator.md        ← 생성자 규칙
│   └── 02_evaluator.md        ← 평가자 루브릭
└── project/                   ← 결과물이 여기에 생성
    ├── index.html
    ├── style.css
    └── app.js
```

Step 2 — Antigravity에게 요청

```
@agents 폴더의 에이전트 지침서를 읽고,
project/ 폴더에 GitHub Issues API 기반의
''한 줄 소감 보드''를 만들어줘.
```

AI 하네싱 황금률 5가지

1. **구체적으로 요청하라**
   "예쁘게" 대신 "글라스모피즘 + 보라색 그라디언트"

AI 하네싱 황금률 5가지

1. **구체적으로 요청하라**
   "예쁘게" 대신 "글라스모피즘 + 보라색 그라디언트"
2. **한 번에 하나씩 요청하라**
   요청이 많으면 쪼개서 전달 — AI도 한 번에 너무 많으면 헷갈립니다

AI 하네싱 황금률 5가지

1. **구체적으로 요청하라**
   "예쁘게" 대신 "글라스모피즘 + 보라색 그라디언트"
2. **한 번에 하나씩 요청하라**
   요청이 많으면 쪼개서 전달 — AI도 한 번에 너무 많으면 헷갈립니다
3. **결과를 직접 확인하라**
   브라우저에서 반드시 테스트 — Antigravity가 대신 확인해주기도 합니다

AI 하네싱 황금률 5가지

1. **구체적으로 요청하라**
   "예쁘게" 대신 "글라스모피즘 + 보라색 그라디언트"
2. **한 번에 하나씩 요청하라**
   요청이 많으면 쪼개서 전달 — AI도 한 번에 너무 많으면 헷갈립니다
3. **결과를 직접 확인하라**
   브라우저에서 반드시 테스트 — Antigravity가 대신 확인해주기도 합니다
4. **틀려도 괜찮다**
   수정 요청으로 반복 개선 — 에이전트 루프가 이를 자동화!

AI 하네싱 황금률 5가지

1. **구체적으로 요청하라**
   "예쁘게" 대신 "글라스모피즘 + 보라색 그라디언트"
2. **한 번에 하나씩 요청하라**
   요청이 많으면 쪼개서 전달 — AI도 한 번에 너무 많으면 헷갈립니다
3. **결과를 직접 확인하라**
   브라우저에서 반드시 테스트 — Antigravity가 대신 확인해주기도 합니다
4. **틀려도 괜찮다**
   수정 요청으로 반복 개선 — 에이전트 루프가 이를 자동화!
5. **기초 지식이 무기다**
   HTML/CSS/JS를 알면 AI 결과물을 더 잘 검증할 수 있습니다

오늘 배운 것 요약

✓ AI 코딩 어시스턴트 생태계 이해 (Copilot, Cursor, Antigravity 등)

✓ Antigravity의 핵심 기능: 파일 + 터미널 + 브라우저 + 이미지 통합

✓ 효과적인 프롬프트 작성법: **CRAP 공식**

✓ 에이전트 루프: Generate → Evaluate → Revise 자동화 사이클

✓ 실제 프로젝트를 AI와 함께 만드는 경험

AI 시대의 개발자 역할

Before (코드를 쓰는 사람)

문법 암기
패턴 학습
반복 타이핑
구글 검색 디버깅

After (방향을 정하는 사람)

무엇을 만들지 결정
품질을 검증
비즈니스 가치 판단
AI 결과물을 **커뮤니케이션**

8주간의 여정이 의미하는 것

Python, 데이터 분석, 웹 개발의 기초를 배운 여러분은 이제 AI의 결과물을 **판단하고 방향을 제시**할 수 있는 역량을 갖추었습니다.

추천 자료

자료	링크
Anthropic 프롬프트 가이드	https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering
GitHub Copilot 문서	https://docs.github.com/en/copilot
바이브 코딩 소개	https://en.wikipedia.org/wiki/Vibe_coding

AI와 함께하는
즐거운 코딩 생활이 되기를!

수고 많으셨습니다.